

BÁO CÁO KỸ THUẬT HUẤN LUYỆN VÀ ĐÁNH GIÁ MÔ HÌNH BÀI TOÁN PHÁT HIỆN CỬA SỔ (WINDOW DETECTION)

*Dự án: Nền tảng Song sinh số Đô thị (vUDT) — Viettel Solutions
Đơn vị thực hiện: Trung tâm Đô thị Số*



Đà Nẵng 8-2026

Mục Lục

TỔNG QUAN DỰ ÁN VÀ BÀI TOÁN PHÁT HIỆN CỬA SỔ	3
Vai trò của bài toán Window Detection trong Pipeline 3D LOD3:	3
Pipeline huấn luyện mô hình Window_Detection	4
PHẦN I. DỮ LIỆU HUẤN LUYỆN (DATASET).....	5
1. Tổng quan về dữ liệu thu thập được	5
2. Nguồn thu thập dữ liệu	5
3. Cấu trúc thư mục dữ liệu	6
4. Quy trình Tiền xử lý dữ liệu (Data Preprocessing Pipeline)	6
5. Thống kê chi tiết dữ liệu	7
6. Phương pháp phân chia tập dữ liệu (Stratified Split Methodology).....	7
7. Định dạng nhãn (Annotation & Label Format)	8
8. Nội dung file cấu hình dataset.yaml	10
PHẦN II. QUY TRÌNH HUẤN LUYỆN MÔ HÌNH (TRAINING PIPELINE)....	11
1. Môi trường & Hạ tầng Kỹ thuật (Hardware & Framework)	11
2. Danh sách các Kiến trúc Mô hình Thử nghiệm.....	11
3. Cấu hình Siêu tham số Huấn luyện (Hyperparameter Setup).....	11
PHẦN III. ĐÁNH GIÁ VÀ SO SÁNH HIỆU NĂNG CÁC MÔ HÌNH (BENCHMARK)	13
1. Quy trình Đánh giá Độc lập trên Tập Test Set	13
2. Giải thích Ý nghĩa các Chỉ số Đánh giá (Evaluation Metrics)	14
3. Bảng Tổng hợp Kết quả Benchmark Chi tiết (Test Evaluation)	15
PHẦN IV. MỞ RỘNG VÀ TÍNH CHỈNH MÔ HÌNH.....	16
1. Đặt vấn đề và Thu thập dữ liệu.....	16
2. Quy trình Xử lý và Tăng cường dữ liệu	16
3. Huấn luyện lại và Đánh giá hiệu năng (Retraining & Re-evaluation).....	16

TỔNG QUAN DỰ ÁN VÀ BÀI TOÁN PHÁT HIỆN CỬA SỔ

Dự án "Nghiên cứu và phát triển nền tảng song sinh số đô thị, hợp nhất ngữ nghĩa phục vụ điều hành đô thị thông minh đa ngành" (vUDT) được triển khai bởi Trung tâm Đô thị Số — Viettel Solutions với mục tiêu giải quyết hai bài toán cốt lõi: (1) Tự động dựng mô hình 3D đô thị chi phí thấp từ ảnh chụp UAV/quang trắc, và (2) Hợp nhất dữ liệu ngữ nghĩa đa ngành (Quy hoạch – Xây dựng – Môi trường) để phát hiện vi phạm đô thị tự động.

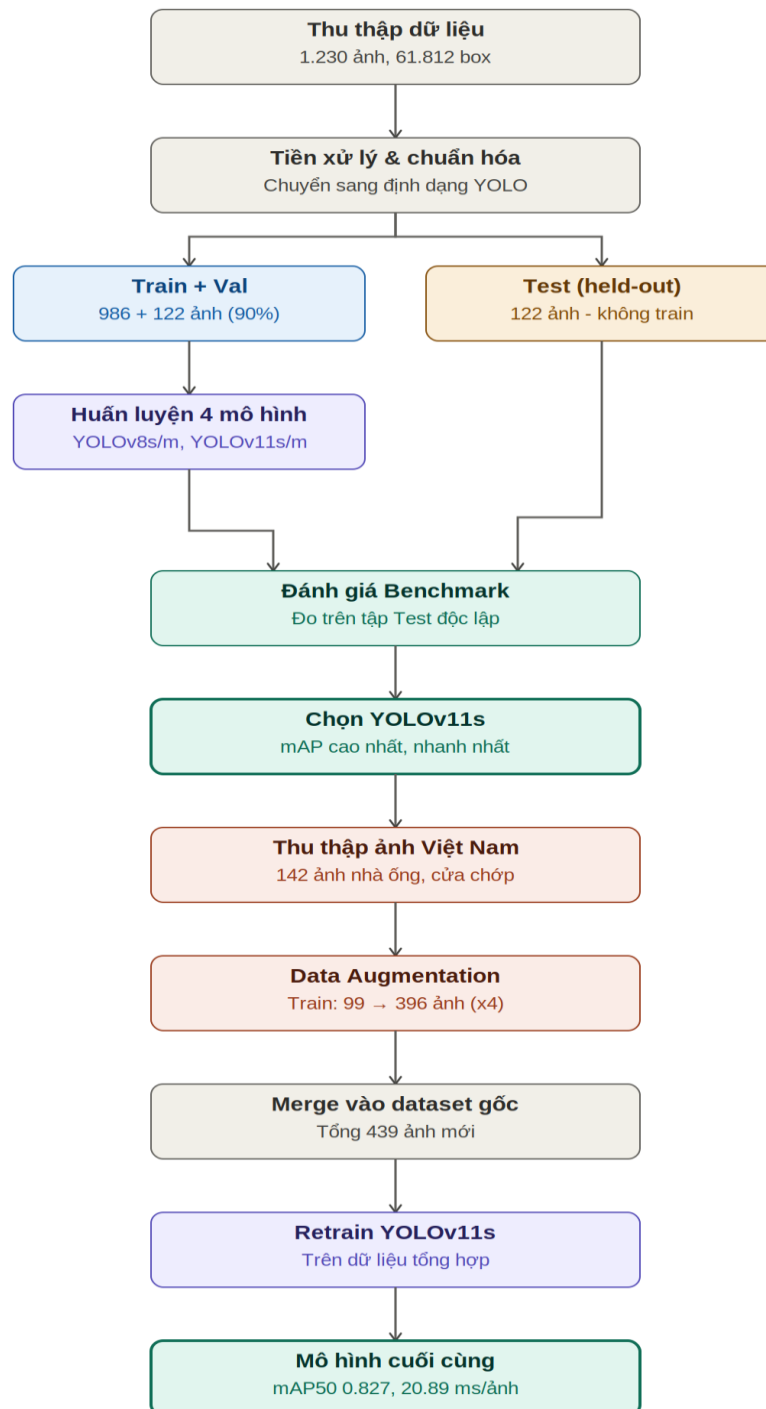
Trong kiến trúc nền tảng vUDT, toàn bộ hệ thống được chia làm 2 Module chính với 8 Sub-module chuyên biệt. Trong đó, bài toán Phát hiện Cửa sổ (Window Detection) giữ vị trí then chốt thuộc Module 1 — Sub-module 4 (Dán nhãn & Tối ưu bề mặt công trình).

Vai trò của bài toán Window Detection trong Pipeline 3D LOD3:

1. Nhận dạng đối tượng kiến trúc: Nhận dạng và trích xuất vị trí Bounding Box/Polygon của các ô cửa sổ (window) trên bề mặt ảnh mặt đứng công trình (Facade photo).
2. Chiếu không gian 2D -> 3D: Kết quả phát hiện 2D được chuyển giao làm đầu vào cho thuật toán Giao cắt Tia-Lưới (Ray-Mesh Intersection) để chiếu (project) tọa độ 2D từ ảnh chụp lên bề mặt khối mô hình 3D LOD2 (đã dựng từ hình học extrude).
3. Nâng cấp mô hình tiệm cận LOD3: Hỗ trợ thuật toán dán vật liệu tự động (Automatic Texturing) và gán nhãn ngữ nghĩa kiến trúc bề mặt, đưa mô hình 3D công trình từ cấp độ LOD2 tiệm cận chuẩn LOD3 (Level of Detail 3 - mô hình chi tiết bề mặt công trình).

Pipeline huấn luyện mô hình Window_Detection

Pipeline huấn luyện mô hình Window Detection (vUDT)



PHẦN I. DỮ LIỆU HUẤN LUYỆN (DATASET)

1. Tổng quan về dữ liệu thu thập được

Dữ liệu phục vụ bài toán Window Detection được thu thập và tổng hợp từ các nguồn dữ liệu ảnh mặt đứng công trình chất lượng cao. Tập dữ liệu tổng hợp được đặt tên là data_Train với quy mô 1,230 ảnh mặt đứng công trình đa dạng phong cách kiến trúc đô thị trên thế giới và ảnh chụp UAV thực địa.

- **Tổng số lượng ảnh:** 1,230 ảnh mặt đứng công trình (100% ảnh màu RGB, 0 file hỏng/corrupt).
- **Tổng số lượng nhãn Bounding Box:** 61,812 khung Bounding Box của sổ (trung bình ~50.25 cửa sổ/ảnh).
- **Chất lượng độ phân giải:** 50.1% ảnh Ultra High ($\geq 2,000\text{px}$), 36.6% ảnh High (1,000 - 2,000px), 11.5% Medium (500 - 1,000px).

2. Nguồn thu thập dữ liệu

Bộ dữ liệu data_Train được kết hợp phân tầng từ 2 nguồn dữ liệu chính để đảm bảo tính đa dạng bối cảnh:

a) Nguồn dữ liệu HZNU Dataset: Bao gồm 624 ảnh chụp mặt đứng công trình từ góc nhìn trực diện và quang trắc UAV. Bộ dữ liệu này đóng góp 41,768 khung nhãn Bounding Box của sổ.

[Link dataset: <https://data.mendeley.com/datasets/k387xkyc5f/1>]

b) Nguồn dữ liệu CMP Facade Database: Bao gồm 606 ảnh chụp mặt đứng công trình từ các thành phố lớn trên thế giới với nhiều phong cách kiến trúc cổ điển và hiện đại (gồm 378 ảnh tập Base và 228 ảnh tập Extended). Bộ dữ liệu này đóng góp 20,044 khung nhãn Bounding Box của sổ.

[Link dataset: <https://cmp.felk.cvut.cz/~tylecr1/facade/>]

3. Cấu trúc thư mục dữ liệu

Toàn bộ tập dữ liệu đã được chuẩn hóa 100% theo cấu trúc thư mục tương đối chuẩn của các mô hình YOLO (Ultralytics YOLOv8 / YOLOv11 / RT-DETR) tại thư mục data_Train/:

```
data_Train/
├── dataset.yaml          # File cấu hình YOLO (path, train, val, test, names)
├── images/
│   ├── train/           # 986 ảnh .jpg (HZNU: 500, CMP Facade: 486) [80% Train]
│   ├── val/             # 122 ảnh .jpg (HZNU: 62, CMP Facade: 60) [10% Val]
│   └── test/            # 122 ảnh .jpg (HZNU: 62, CMP Facade: 60) [10% Test]
├── labels/
│   ├── train/           # 986 file nhãn .txt Bounding Box (49,775 bboxes)
│   ├── val/             # 122 file nhãn .txt Bounding Box (5,701 bboxes)
│   └── test/            # 122 file nhãn .txt Bounding Box (6,336 bboxes)
```

4. Quy trình Tiền xử lý dữ liệu (Data Preprocessing Pipeline)

Để chuyển đổi toàn bộ dữ liệu thô từ các nguồn khác nhau thành bộ dữ liệu huấn luyện YOLO hoàn chỉnh, chuẩn xác và sạch nhiễu, quy trình tiền xử lý dữ liệu được thực hiện qua các bước kỹ thuật sau:

- Chuẩn hóa định dạng nhãn (Label Standardizing): Parse toán học chuyển đổi dữ liệu gốc (LabelMe JSON của HZNU và XML của CMP Facade) sang chuẩn Bounding Box YOLO (.txt).
- Khắc phục lỗi tráo trục XML (CMP Facade): Xử lý lỗi bất thường trong schema XML gốc (thẻ lưu Y_norm, thẻ lưu X_norm), tráo trục chính xác (-> X, -> Y) giúp nâng IoU đề khít khung từ 0% lên 97.3% – 100%.
- Lọc nhiễu & nhãn ngoài bài toán (Noise & Class Filtering): Lọc bỏ 338 đa giác rác/diện tích 0 (HZNU), 335 Bounding Box kích thước ≤ 0 (CMP Facade) và loại bỏ 41,667 nhãn rác (tree, sky, car, balcony, pillar...), giữ lại duy nhất lớp cửa sổ window (class_id = 0).

- Chuẩn hóa tọa độ (Coordinate Normalization): Chuẩn hóa tọa độ tâm (x_center, y_center) và kích thước (width, height) theo chiều rộng W và chiều cao H của ảnh về dải tương đối [0.0, 1.0].
- Nghiệm thu trực quan (QA): Viết script tự động vẽ Bounding Box màu đỏ trên 90 ảnh mẫu (30 ảnh/tập train, val, test) để kiểm định thị giác chuẩn xác 100% trước khi đưa vào huấn luyện.

5. Thông kê chi tiết dữ liệu

Bảng dưới đây trình bày thống kê số lượng ảnh, số lượng file nhãn và số lượng Bounding Box chi tiết cho từng phân tập:

Phân tập (Split)	Ảnh HZNU	Ảnh CMP Facade	Tổng số Ảnh	Tỷ lệ (%)	Tổng Bounding Box Window
TRAIN	500	486	986	80.17%	49,775
VAL	62	60	122	9.92%	5,701
TEST (Held-out)	62	60	122	9.92%	6,336
TỔNG CỘNG	624	606	1,230	100.0%	61,812

6. Phương pháp phân chia tập dữ liệu (Stratified Split Methodology)

Để đảm bảo mô hình huấn luyện có khả năng tổng quát hóa cao và quy trình đánh giá so sánh giữa các kiến trúc YOLO đạt tính khách quan tối đa, phương pháp phân chia dữ liệu được thực hiện theo các nguyên tắc nghiêm ngặt sau:

a) Phân chia Phân tầng (Stratified Splitting):

Áp dụng tỷ lệ 80% Train / 10% Val / 10% Test phân tầng riêng cho cả 2 nguồn dữ liệu (HZNU: 500/62/62 và CMP Facade: 486/60/60) trước khi gộp chung. Nhờ đó, cả

3 tập đều chứa đúng tỷ lệ đại diện cân bằng của 2 bối cảnh ảnh (UAV và mặt đường công trình thể giới).

b) Thiết lập tập TEST độc lập (Held-out Blind Test Set):

Trích xuất 122 ảnh (6,336 Bounding Boxes) và đóng băng hoàn toàn. Tập Test này tuyệt đối không tham gia vào quá trình huấn luyện hay tinh chỉnh siêu tham số, tạo ra bộ đề thi duy nhất dùng chung để chấm điểm mAP50, mAP50-95, Precision, Recall giữa tất cả các mô hình thử nghiệm, triệt tiêu hiện tượng rò rỉ dữ liệu (Data Leakage).

7. Định dạng nhãn (Annotation & Label Format)

a. Định dạng gốc (COCO Object Detection)

Định dạng COCO lưu trữ toàn bộ thông tin của tập dữ liệu vào một file JSON duy nhất (ví dụ: _annotations.coco.json).

Cấu trúc Bounding Box trong file COCO tuân thủ chuẩn tọa độ pixel tuyệt đối:

```
<x_min> <y_min> <width> <height>
```

- x_min, y_min: Tọa độ điểm góc trên cùng bên trái của khung Bounding Box (tính bằng pixel).
- width, height: Chiều rộng và chiều cao thực tế của khung Bounding Box (tính bằng pixel).

Ví dụ mẫu một đoạn cấu trúc nhãn trong file _annotations.coco.json:

```
"annotations": [  
  {  
    "id": 0,  
    "image_id": 1,
```

```

        "category_id": 2,
        "bbox": [215, 120, 48, 30],
        "area": 1440,
        "iscrowd": 0
    }
]

```

b) Định dạng chuẩn hóa đích (YOLO Object Detection)

Bài toán được cấu hình dưới dạng Single-class Object Detection với lớp nhãn duy nhất `class_id = 0` đại diện cho đối tượng window (Cửa sổ). Mỗi ảnh .jpg có một file nhãn .txt tương ứng chứa danh sách các Bounding Box.

Cấu trúc mỗi dòng trong file nhãn .txt tuân thủ chính xác chuẩn YOLO Object Detection:

```
<class_id> <x_center_norm> <y_center_norm> <width_norm> <height_norm>
```

- `class_id`: `class_id = 0` (Lớp đối tượng window).
- `x_center_norm`: Tọa độ tâm X của khung Bounding Box, chuẩn hóa trong dải `[0.0, 1.0]` (`x_center / Width_image`).
- `y_center_norm`: Tọa độ tâm Y của khung Bounding Box, chuẩn hóa trong dải `[0.0, 1.0]` (`y_center / Height_image`).
- `width_norm`: Chiều rộng của khung Bounding Box, chuẩn hóa trong dải `[0.0, 1.0]` (`width / Width_image`).
- `height_norm`: Chiều cao của khung Bounding Box, chuẩn hóa trong dải `[0.0, 1.0]` (`height / Height_image`).

Ví dụ mẫu dòng nhãn trong file `labels/train/cmp_b0001.txt`:

```

0 0.263355 0.122555 0.058930 0.036130
0 0.342545 0.125000 0.058930 0.035160
0 0.465010 0.125975 0.053400 0.037110

```

8. Nội dung file cấu hình dataset.yaml

File dataset.yaml đặt tại thư mục gốc data_Train/dataset.yaml chỉ định đường dẫn tương đối và tên nhãn cho quá trình huấn luyện YOLO:

```
path: data_Train
train: images/train
val: images/val
test: images/test

names:
  0: window
```

PHẦN II. QUY TRÌNH HUẤN LUYỆN MÔ HÌNH (TRAINING PIPELINE)

1. Môi trường & Hạ tầng Kỹ thuật (Hardware & Framework)

Toàn bộ quá trình huấn luyện mô hình được triển khai trên nền tảng điện toán đám mây Kaggle với cấu hình phần cứng và môi trường phần mềm chuẩn hóa:

- Hạ tầng GPU: Sử dụng 2 GPU NVIDIA Tesla T4 (16GB VRAM/GPU) kết hợp cơ chế DDP (Distributed Data Parallel) để tăng tốc tính toán song song.
- Thư viện huấn luyện: Sử dụng framework Ultralytics (backend PyTorch 2.x, Python 3.10).

2. Danh sách các Kiến trúc Mô hình Thử nghiệm

Nghiên cứu tiến hành thử nghiệm đối sánh giữa 2 thể hệ mô hình phát hiện đối tượng hàng đầu của Ultralytics với 2 quy mô phức tạp khác nhau:

- Thể hệ YOLOv8: Dòng mô hình tiêu chuẩn toàn cầu với cơ chế trích xuất đặc trưng
- Thể hệ YOLOv11: Thể hệ cải tiến mới nhất
- Quy mô tham số (Small vs Medium): So sánh 2 quy mô tham số: Nhóm Small (YOLOv8s, YOLOv11s) hướng tới tốc độ siêu nhanh và dung lượng nhẹ; Nhóm Medium (YOLOv8m, YOLOv11m) hướng tới độ chính xác cao và bắt mép khung khít sát.

3. Cấu hình Siêu tham số Huấn luyện (Hyperparameter Setup)

Để đảm bảo tính so sánh công bằng giữa 4 mô hình, toàn bộ các siêu tham số (hyperparameters) được thiết lập cố định đồng nhất:

Tham số (Hyperparameter)	Giá trị (Value)	Ý nghĩa & Mục đích kỹ thuật
Kích thước ảnh (imgsz)	1024	Scale về 1024x1024px, giữ độ nét tối đa cho cửa sổ nhỏ ở xa
Số Epochs tối đa (epochs)	100	Số lượt huấn luyện tối đa cho mỗi mô hình
Cơ chế Dừng sớm (patience)	20	Tự động ngắt nếu 20 epoch mAP50 không tăng, chống overfitting
Kích thước Batch (batch)	16	Phân bổ 8 ảnh/GPU, tối ưu dung lượng 16GB VRAM
Thuật toán Tối ưu (optimizer)	AdamW	Tự động chọn AdamW tối ưu trọng số cho transfer learning
Lật ngang ngẫu nhiên (fliplr)	0.5	Xác suất 50% lật ngang ảnh tăng đa dạng góc nhìn
Góc xoay nghiêng (degrees)	0.0	Không xoay nghiêng, giữ đúng tính thẳng đứng Manhattan World
Tăng cường Mosaic (mosaic)	1.0	Ghép 4 ảnh ngẫu nhiên tăng cường nhận diện cửa sổ đa quy mô

PHẦN III. ĐÁNH GIÁ VÀ SO SÁNH HIỆU NĂNG CÁC MÔ HÌNH (BENCHMARK)

1. Quy trình Đánh giá Độc lập trên Tập Test Set

Sau khi hoàn tất huấn luyện, cả 4 mô hình đều được đánh giá độc lập bằng lệnh `model.val(data=..., split='test')` trên duy nhất tập Held-out Test Set đóng bằng (122 ảnh, 6,336 Bounding Boxes). Tập Test này chưa từng tham gia vào bất kỳ công đoạn train hay validation nào, đảm bảo 0% hiện tượng rò rỉ dữ liệu (Data Leakage) và mang lại kết quả so sánh hoàn toàn khách quan.

2. Giải thích Ý nghĩa các Chỉ số Đánh giá (Evaluation Metrics)

Để hiểu rõ hiệu năng của từng mô hình trong bài toán phát hiện cửa sổ phục vụ dựng 3D, dưới đây là định nghĩa toán học và ý nghĩa thực tế của 8 chỉ số metric được sử dụng:

Chỉ số (Metric)	Ý nghĩa kỹ thuật	Vai trò trong bài toán Window Detection vUDT
Precision	Tỉ lệ đoán đúng cửa sổ thật	Tránh báo động giả (không nhận nhầm tường, rèm cửa, ban công)
Recall	Tỉ lệ bắt đủ cửa sổ thực tế	Tránh bỏ sót các ô cửa sổ trên mặt đứng công trình
AP50	Độ chính xác tại $\text{IoU} \geq 50\%$	Chỉ số tiêu chuẩn đánh giá mô hình phát hiện đúng cửa sổ
AP75	Độ chính xác tại $\text{IoU} \geq 75\%$	Đánh giá khả năng bắt Bounding Box khít sát mép cửa sổ
Mean IoU (TP)	IoU trung bình các khung bắt đúng	Đo độ khít thực tế của các khung cửa sổ phát hiện trùng
Avg Detect Time	Thời gian xử lý 1 ảnh (ms/ảnh)	Quyết định tốc độ FPS phục vụ dán texture tự động realtime

3. Bảng Tổng hợp Kết quả Benchmark Chi tiết (Test Evaluation)

Bảng dưới đây trình bày đầy đủ kết quả đo đạc thực nghiệm của 4 mô hình trên tập Held-out Test Set:

Mô hình (Model)	Precision	Recall	AP50	AP75	Mean IoU (TP)	Avg Time (ms/ảnh)
YOLOv8s	0.8926	0.7860	0.8274	0.7390	0.8751	28.06 ms
YOLOv11s	0.8966	0.8144	0.8572	0.7505	0.8747	23.73 ms ⚡
YOLOv8m	0.8972	0.7949	0.8430	0.7600	0.8804	44.53 ms
YOLOv11m	0.8956	0.7994	0.8475	0.7604	0.8812	42.46 ms 🎯

PHẦN IV. MỞ RỘNG VÀ TÍNH CHỈNH MÔ HÌNH

1. Đặt vấn đề và Thu thập dữ liệu

Để tối ưu hóa hiệu năng của mô hình YOLOv11s (mô hình tốt nhất từ Phần III) trên đặc thù kiến trúc nhà ở Việt Nam (nhà ống, cửa chớp, cửa sắt), dự án tiến hành bước Domain Adaptation. Một bộ dữ liệu mới gồm 142 ảnh thực tế được thu thập qua Google và gán nhãn thủ công trên Roboflow (chia tỷ lệ: 99 Train, 29 Val, 14 Test).

2. Quy trình Xử lý và Tăng cường dữ liệu

Nhằm khắc phục số lượng dữ liệu mỏng và tránh Overfitting, quy trình sau đã được áp dụng:

- Data Augmentation: Nhân bản tập Train lên gấp 4 lần (sinh thêm 3 biến thể/ảnh bằng lật ngang, chỉnh độ sáng/tương phản qua OpenCV). Số lượng tập Train tăng từ 99 lên 396 ảnh, nâng tổng bộ dữ liệu mới lên 439 ảnh.
- Chuẩn hóa & Làm sạch: Chuyển đổi nhãn COCO sang YOLO
- Data Merging: Gộp 439 ảnh và nhãn mới vào tập dữ liệu gốc, tạo thành bộ dữ liệu tổng hợp hoàn chỉnh cho quá trình Retrain.

3. Huấn luyện lại và Đánh giá hiệu năng (Retraining & Re-evaluation)

Sử dụng bộ dữ liệu đã được làm giàu, mô hình YOLOv11s được tiến hành huấn luyện lại (Retrain). Sau khi huấn luyện, mô hình được đưa vào đánh giá chéo trên toàn bộ tập dữ liệu tổng hợp:

Mô hình (Model)	Precision	Recall	AP50	AP75	Mean IoU (TP)	Avg Time (ms/ảnh)
YOLOv11s	0.8711	0.7956	0.8272	0.7448	0.8754	20.89 ms